

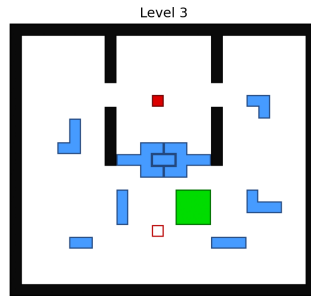
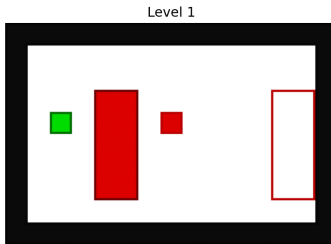
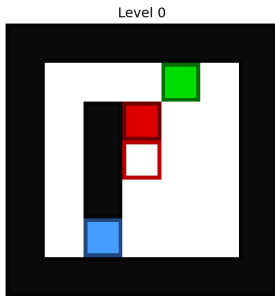
Промежуточный отчёт по PushWorld

Оптимизационный анализ пайплайна обучения

Иванкин Евгений, Сулиман Адам

Что такое PushWorld

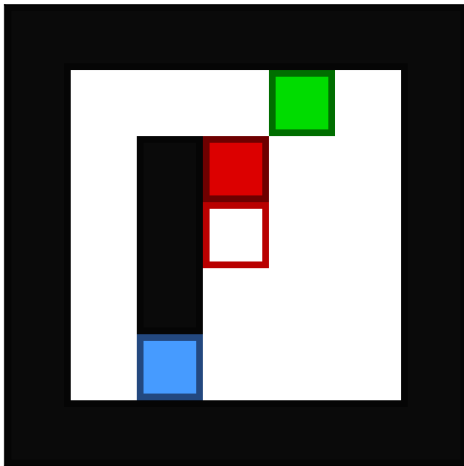
PushWorld benchmark examples



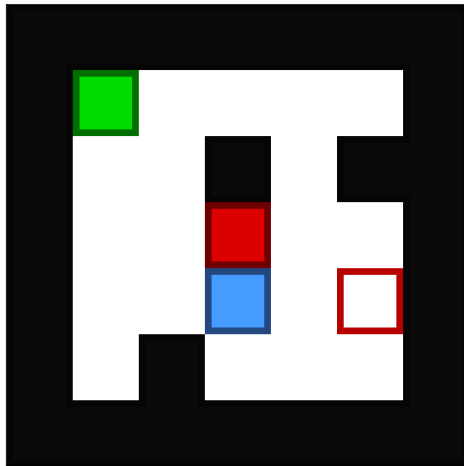
- PushWorld — planning benchmark: агент двигается по сетке и толкает объекты в целевые позиции.
- На сложных уровнях появляются инструменты, узкие проходы и длинные причинно-следственные цепочки.
- Для нас это удобный тест на то, где RL-пайплайн ломается по времени и по качеству.

Как выглядит решение в среде

DQN success
step=0 reward=0.00



PPO stochastic failure
step=0 reward=0.00



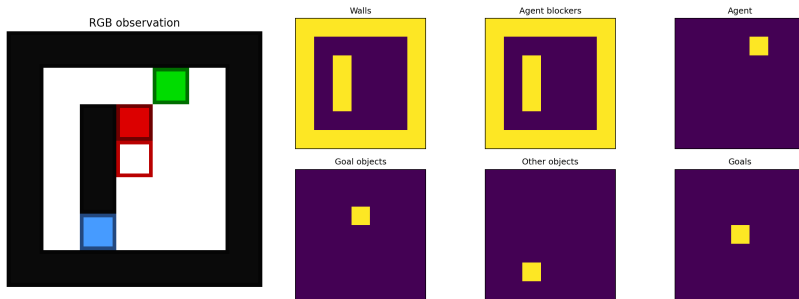
Бейзлайны слабые и медленные

- PPO на RGB оказался нестабильным даже на 5 простых головоломках.
- DQN на том же наборе вел себя устойчивее, но работал очень медленно.
- Это были короткие прогоны порядка 100–200k итераций: по качеству это ещё не финальный вердикт, но слабость бейзлайнов уже видна.
- Для этого checkpoint важнее именно системный сигнал: куда уходит время и какой FPS даёт каждая конфигурация.

Конфигурация	FPS	Успех
PPO, RGB, 5 головоломок	195	15%
DQN, RGB, 5 головоломок	8	40%

Что видит агент: RGB против planes

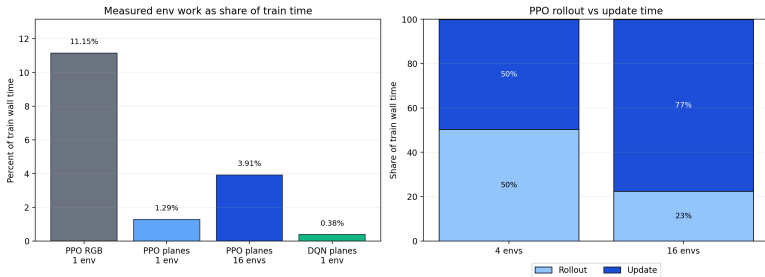
Structured plane observations



- RGB требует рендерить большую картинку и гонять большие тензоры через replay/CNN.
- Plane-наблюдения оставляют только полезную структуру состояния.
- Ключевые числа: размер состояния $58,800 \text{ B} \rightarrow 1,176 \text{ B}$; PPO $195 \rightarrow 586 \text{ FPS}$; DQN $8 \rightarrow 161.7 \text{ FPS}$.

После planes среда уже не главный bottleneck

PushWorld Bottleneck Summary



- PPO, RGB: среда занимает **11.15%** train time. PPO, planes: только **1.29%**.
- PPO, planes + 16 envs: **77.4%** времени уже уходит в update. DQN, planes: среда занимает всего **0.38%**, update — **76.0%**.
- Итог: сейчас главный bottleneck — не симуляция среды, а learner/update stack. Поэтому переписывать среду на GPU прямо сейчас преждевременно.

Ещё одна оптимизация: `torch.compile`

- Мы проверили уже не только representation-level оптимизации, но и model-side оптимизацию.
- `torch.compile` заметно ускоряет чистый forward pass.
- Но end-to-end training step ускоряется совсем слабо.
- Значит, оставшийся bottleneck шире, чем просто скорость инференса.

Batch	Метрика	Eager	Compile
256	Infer/s	1055.6	1643.6
256	Train/s	345.7	359.1
512	Infer/s	614.5	878.7
512	Train/s	201.6	208.3

Ускорение inference: **1.43x–1.56x**
Ускорение training: только **1.03x–1.04x**

Что дальше

Ближайший шаг 1

Goal-conditioned observations + relabeling [1]. Это прямой способ решить проблему sparse-reward, сохраняя уже быстрый plane-pipeline.

Ближайший шаг 2

AMP, настройка update-параметров, более детальный profiling. Это логично, потому что и PPO, и DQN уже упираются не в среду, а в update-часть обучения.

Среднесрочно

Transformer policy на траекториях планировщика [2]. Для planning benchmark sequence-модель выглядит здесь естественно.

Долгосрочно

Гибридный solver: planner + learned executor [3]. Планировщик может задавать порядок подзадач, subgoals или грубый план, а политика — устойчиво исполнять локальные шаги между ними.

- [1] M. Andrychowicz et al. *Hindsight Experience Replay*. NeurIPS, 2017.
arxiv.org/abs/1707.01495 к плану
- [2] T. Wheeler. Серия материалов о Sokoban, sequence modeling и GPU rollout.
timallanwheeler.com/blog/category/sokoban/ к плану
- [3] *Learn to Follow: Decentralized Lifelong Multi-agent Pathfinding via Planning and Learning*, 2023.
arxiv.org/abs/2310.01207 к плану